Tasks on Terraform-05

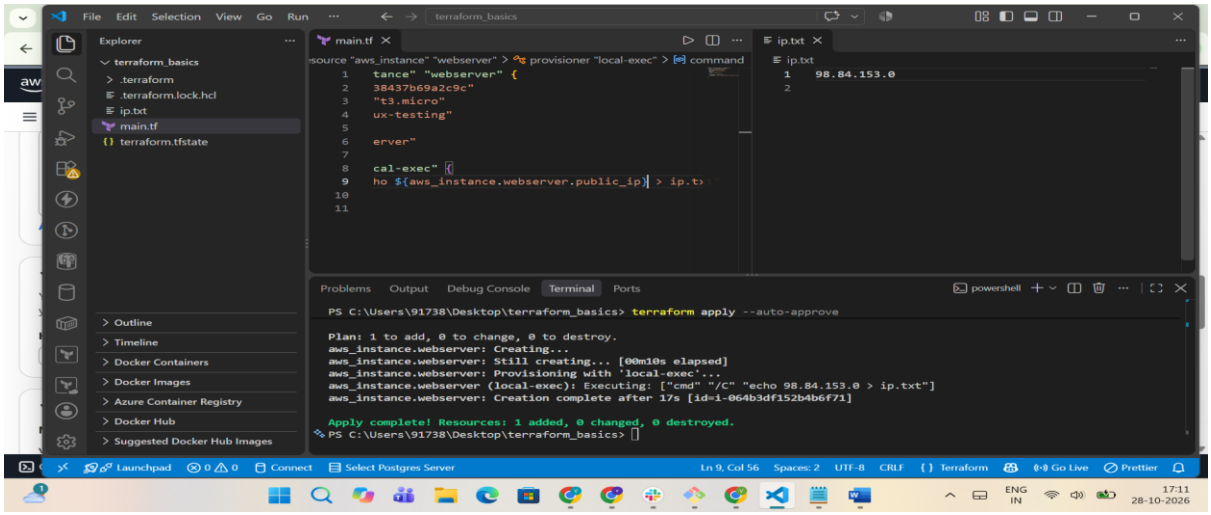
1). Watch the Terraform-05 video.

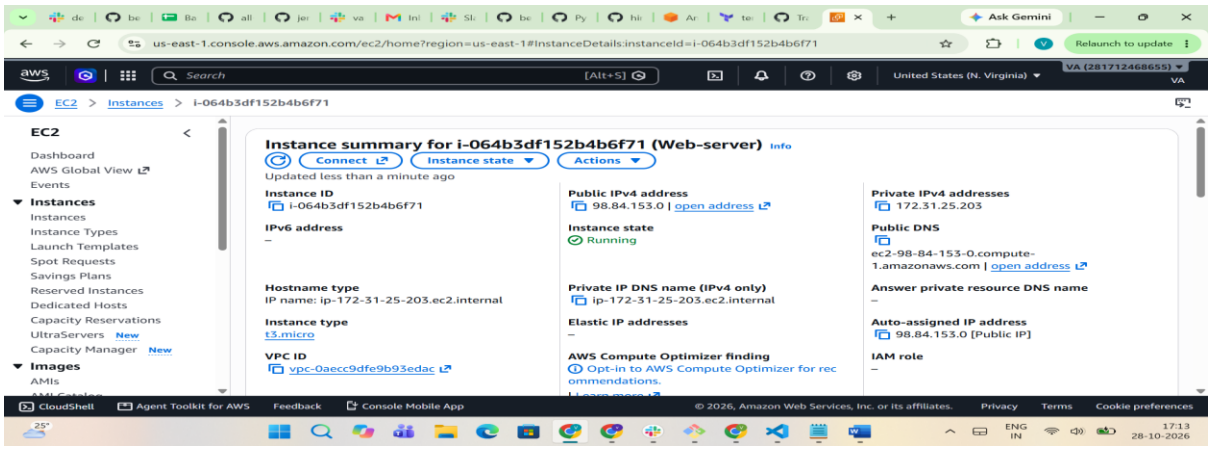
Terraform-05 video completed.

2). Execute the script shown in the video.

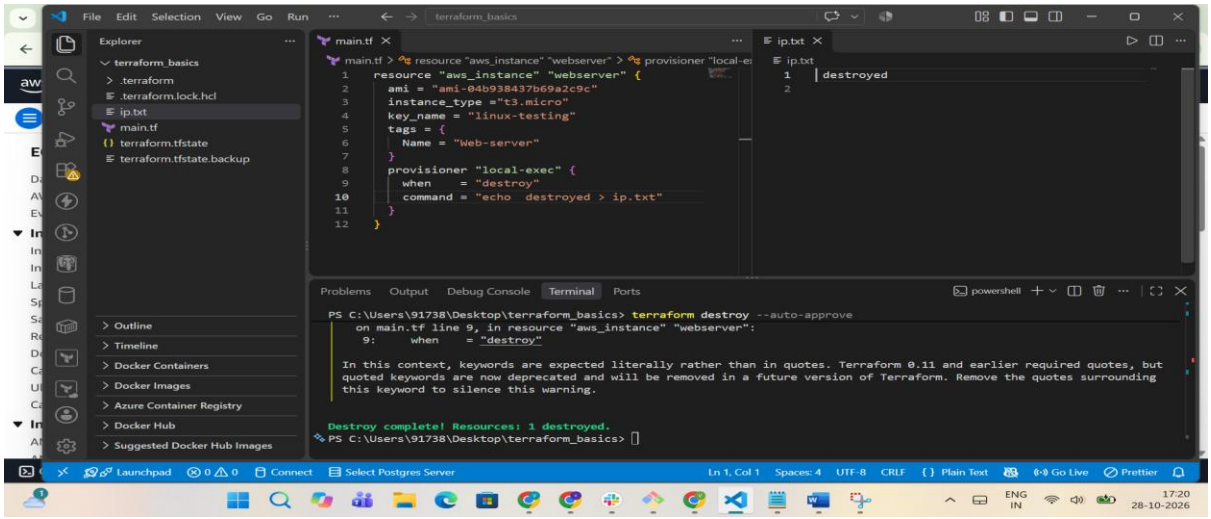
Implementation steps:

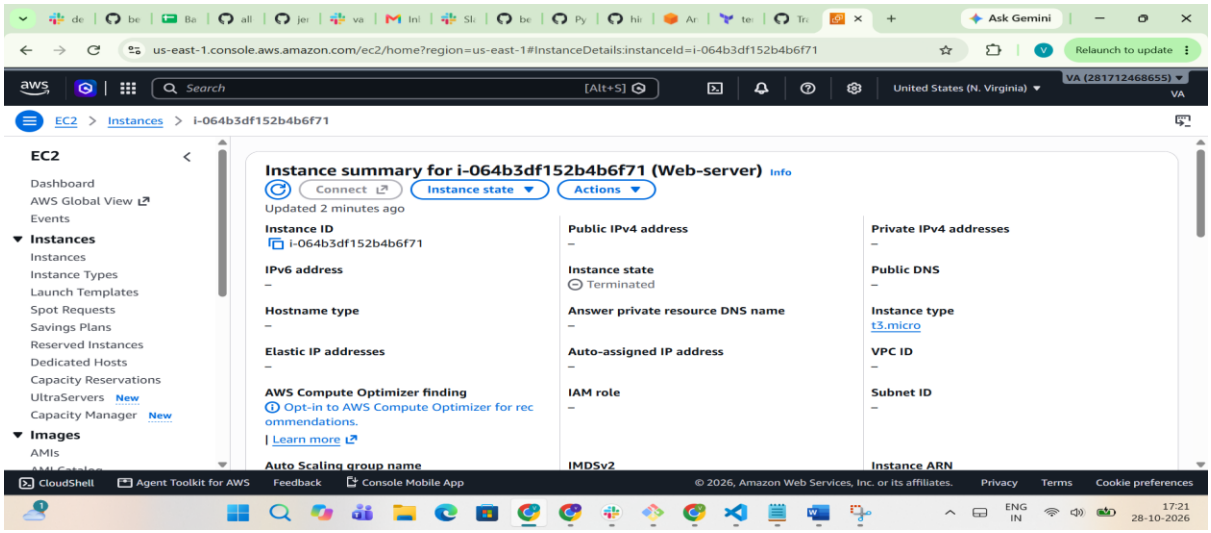
1.provisioner local:



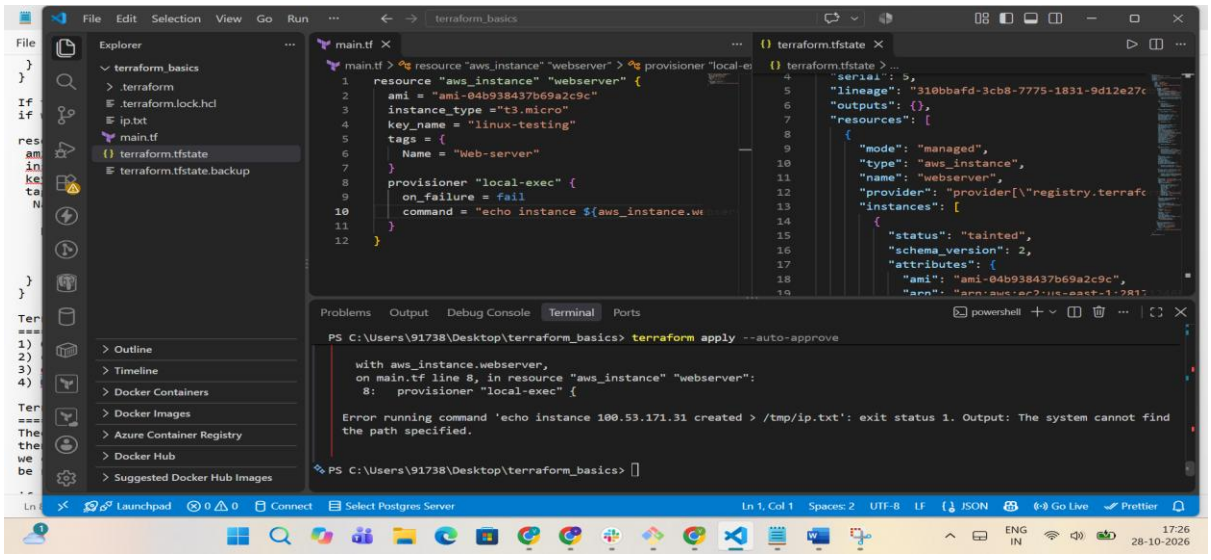


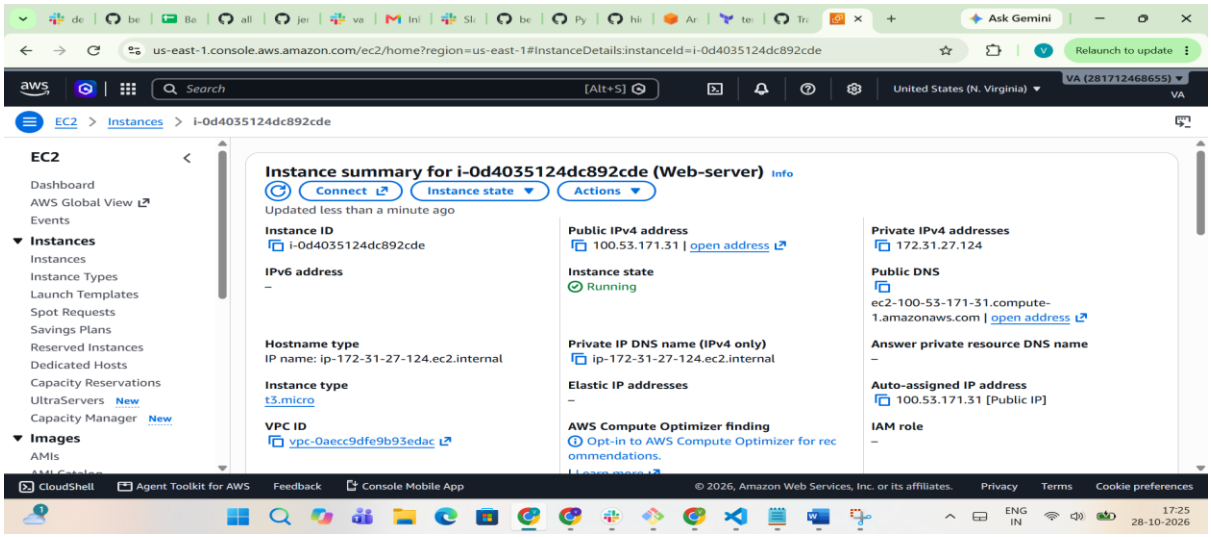
2. Terraform provisioner behaviours: when: “destroy”



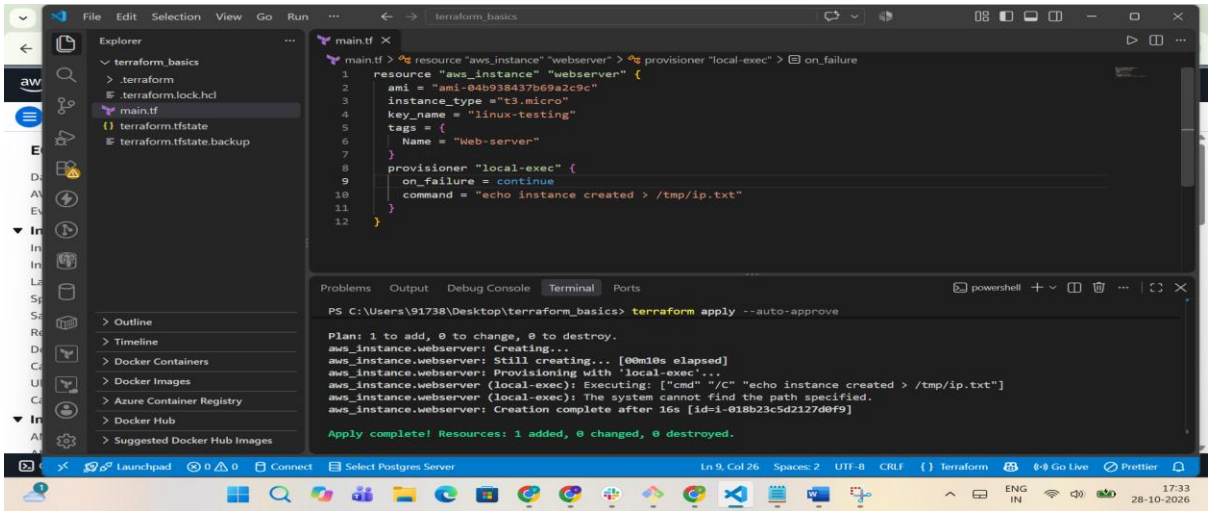


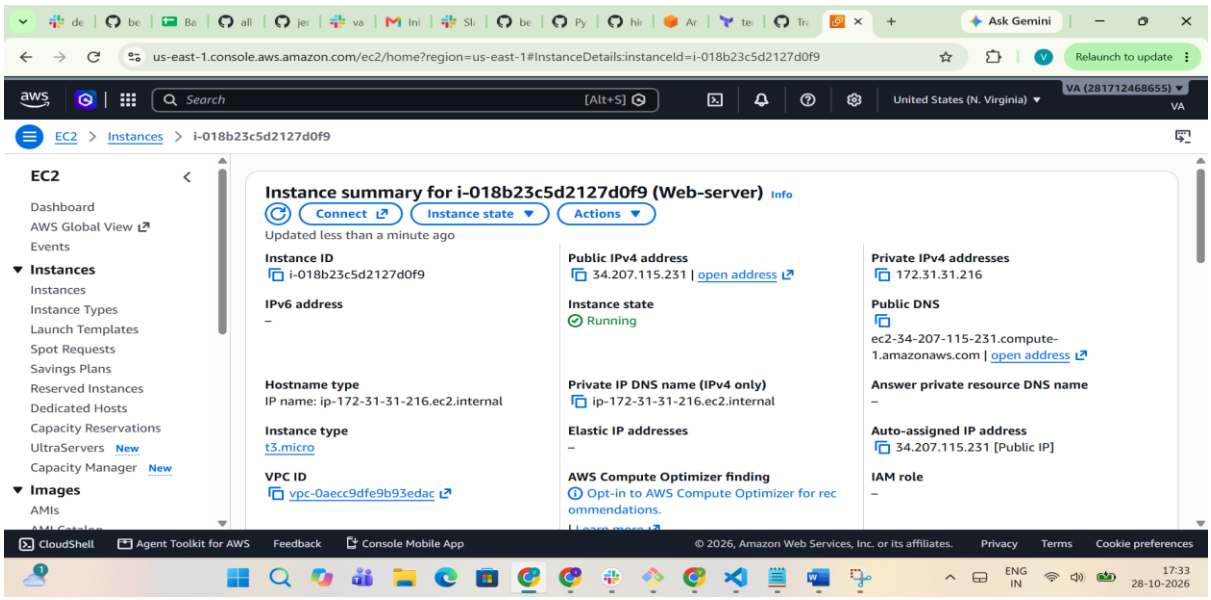
On_failure = fail



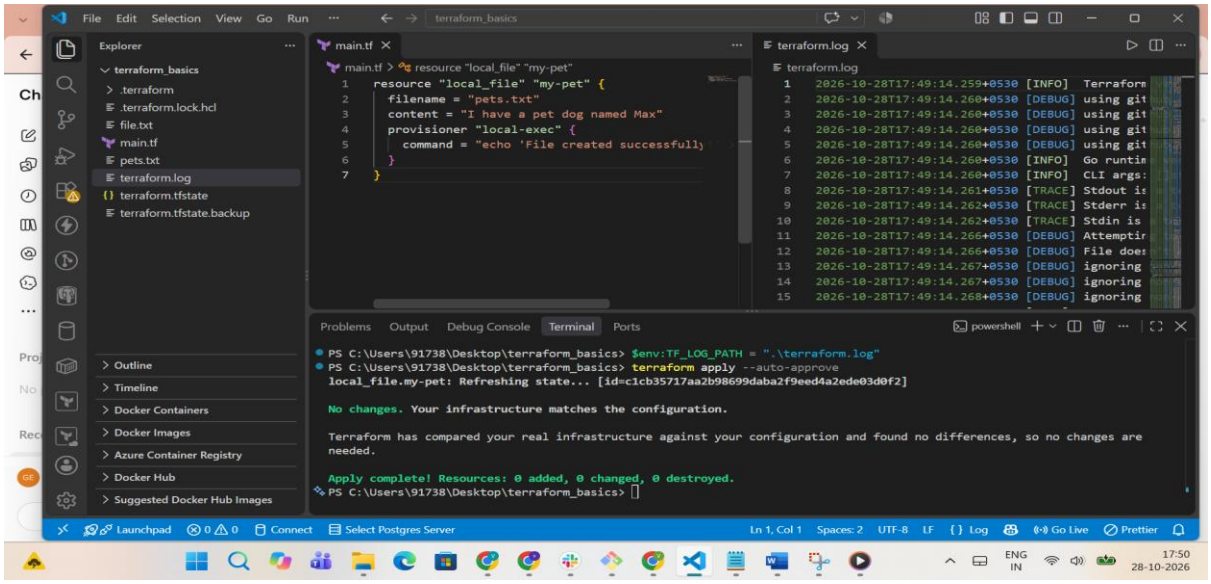


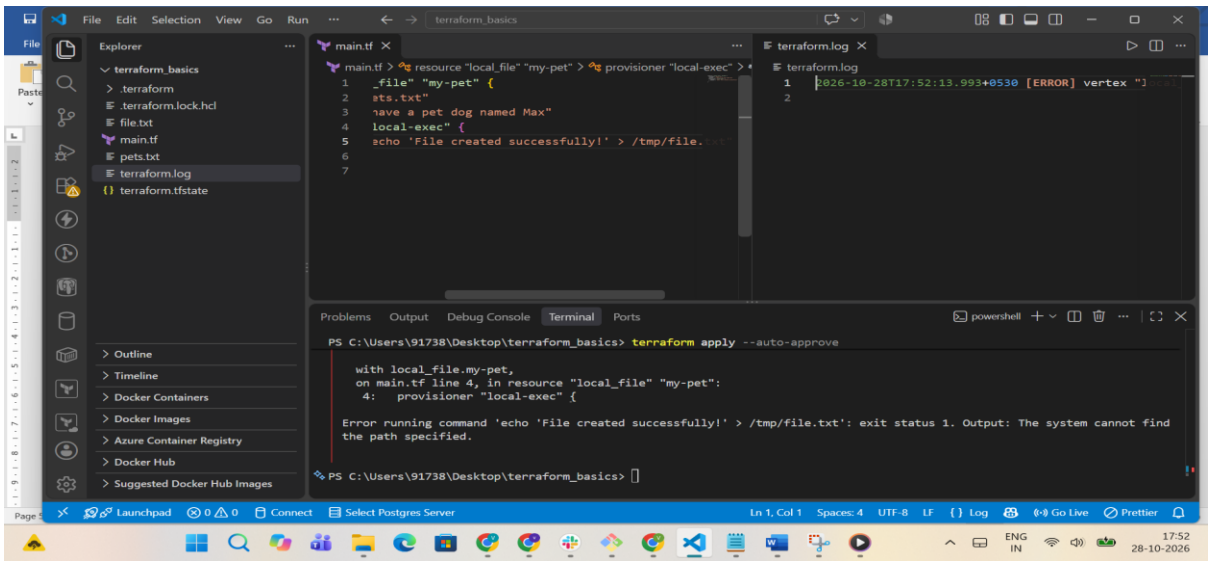
On_failure = continue



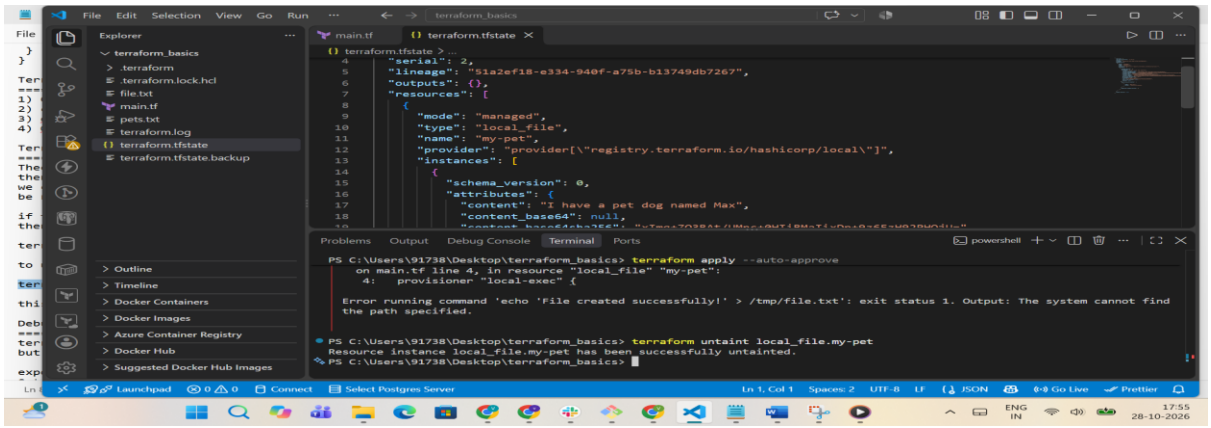


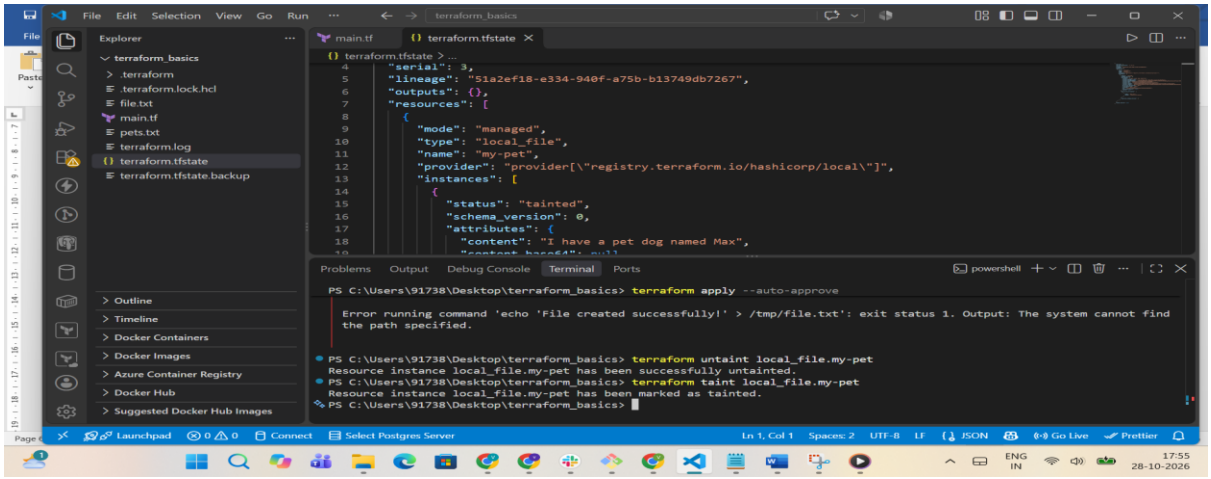
3.Debugging:





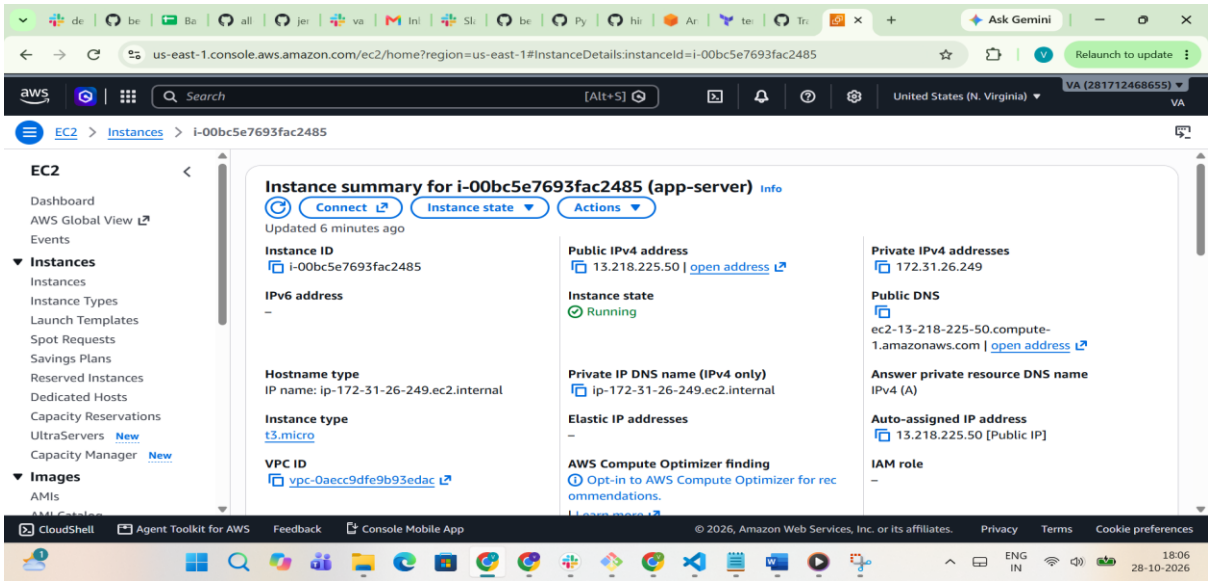
4.Terraform taint and untaint:

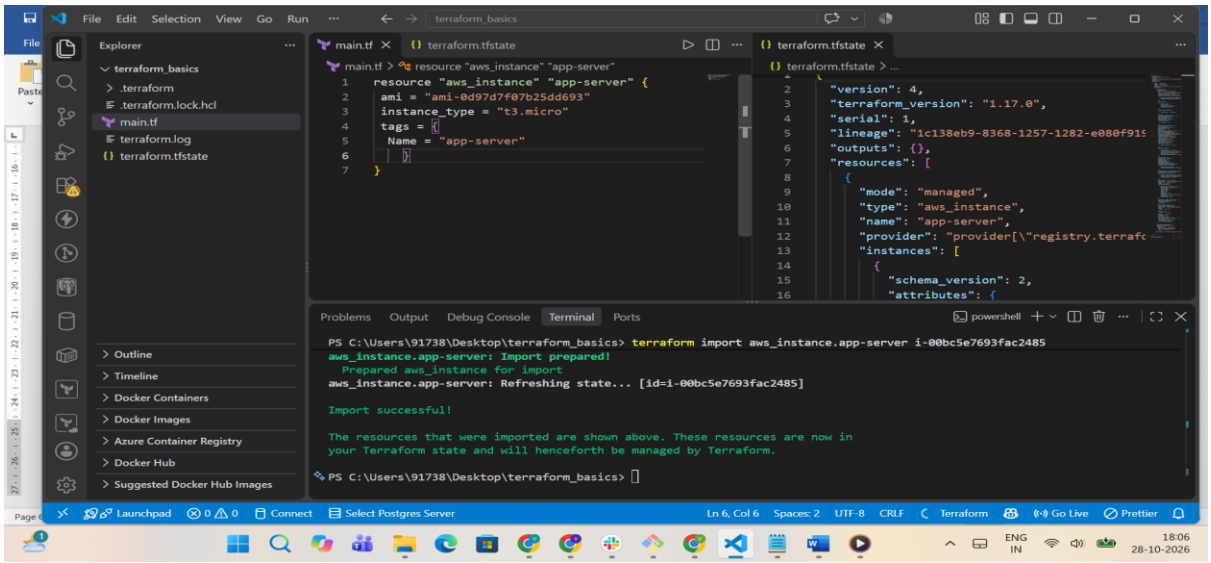




5.Terraform import:

terraform import aws_instance.app-server i-00bc5e7693fac2485





3). Create one EC2 instance with httpd installed using a Terraform script.

Implementation steps:

1. Installed Terraform on the local Windows machine and configured AWS credentials.
2. Created a project folder named terraform_basics.
3. Created a main.tf file containing the AWS provider, AMI data source, security group, EC2 instance and output configurations.
4. Configured the AWS provider to use the us-east-1 region.
5. Retrieved the latest Amazon Linux 2023 AMI using a Terraform data source.
6. Created a security group allowing inbound HTTP traffic on port 80.
7. Created an EC2 instance using the t2.micro instance type.
8. Used the EC2 user_data script to install Apache (httpd), enable and start the service, and create a custom HTML page.
9. Executed terraform init to initialize the working directory.
10. Executed terraform apply and confirmed the deployment.
11. Retrieved the public IP address and website URL from Terraform outputs.
12. Opened the website URL in a browser and verified that the custom text was displayed successfully.

Code — main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
```

```
  region = "us-east-1"
}

data "aws_vpc" "default" {
  default = true
}


resource "aws_security_group" "web_sg" {
  name        = "terraform-http-sg"
  description = "Allow HTTP traffic"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web_server" {
```

```
  ami                    = “ami-04b938437b69a2c9c”
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    dnf update -y
    dnf install -y httpd
    systemctl enable httpd
    systemctl start httpd
    echo "<h1>Apache Server Created Using Terraform</h1>" > /var/www/html/index.html
  EOF

  tags = {
    Name = "Terraform-HTTPD-Server"
  }
}

output "instance_public_ip" {
  value = aws_instance.web_server.public_ip
}

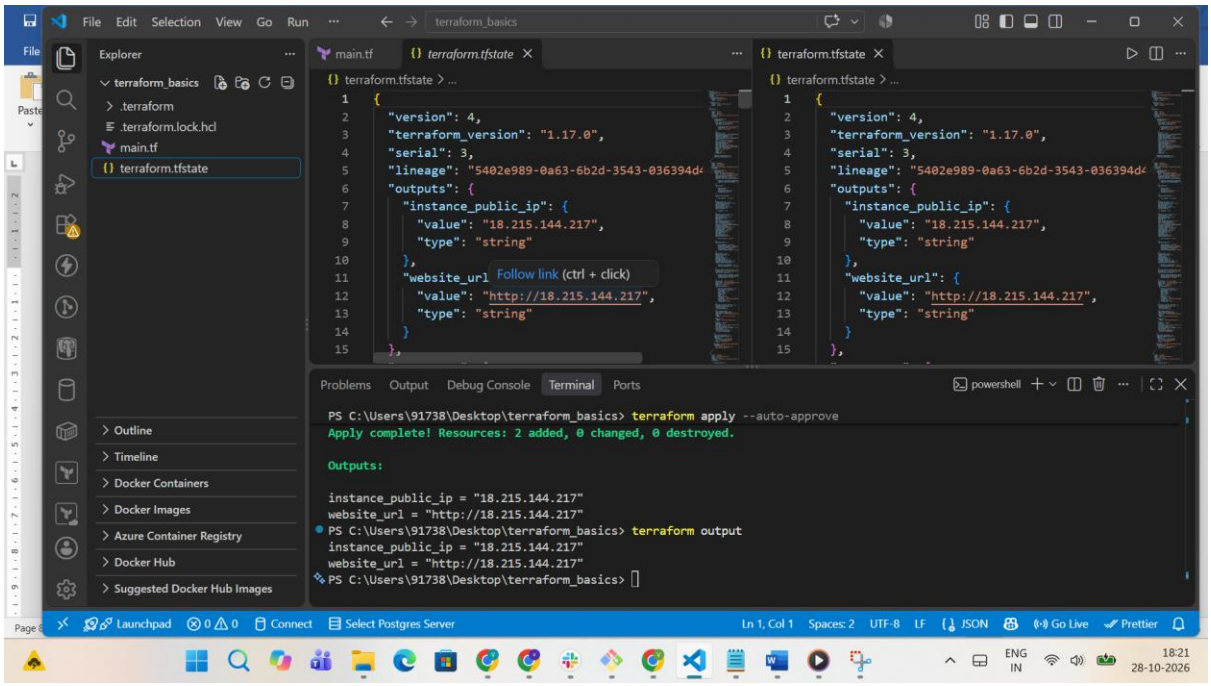
output "website_url" {
  value = "http://${aws_instance.web_server.public_ip}"
}
```

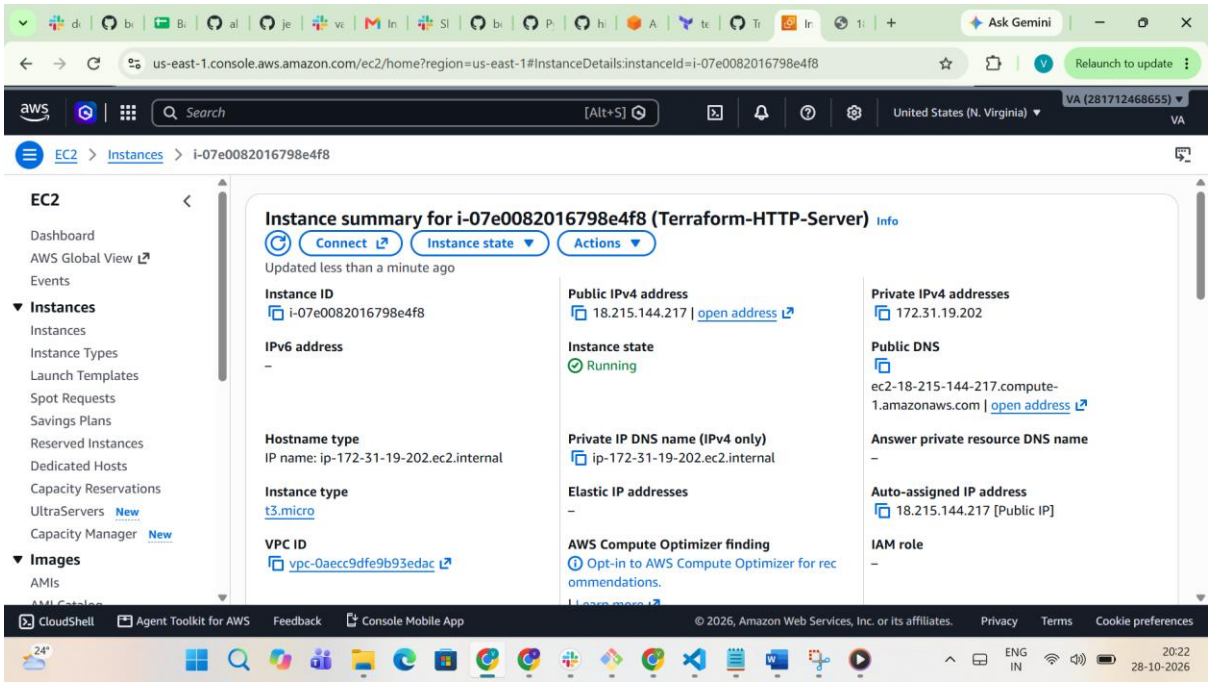
Commands executed

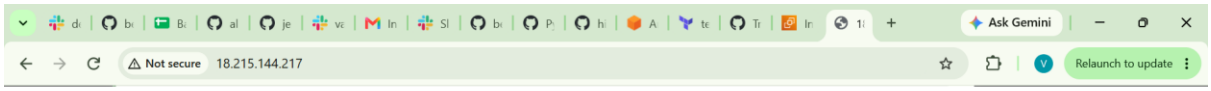
terraform init

terraform apply

Result: The EC2 instance was created successfully, Apache was installed, and the custom webpage was verified in the browser.







Apache Server Created Using Terraform



4). Set up **S3 as backend** for task 3.

Implementation steps:

1. Created an S3 bucket in the us-east-1 region to store the Terraform state file.
2. Enabled S3 bucket versioning to maintain previous versions of the state file.
3. Enabled server-side encryption and blocked public access to the bucket.
4. Used a separate initial configuration to create the S3 bucket and DynamoDB table before configuring the backend.
5. Opened the existing terraform_basics\main.tf file used in Question 3.
6. Added the S3 backend configuration to the existing Terraform block without changing the EC2 resources.
7. Configured the S3 bucket name, state file key and AWS region.
8. Executed terraform init -migrate-state to migrate the existing local Terraform state into S3.
9. Executed terraform validate and terraform plan to verify the configuration and existing infrastructure.
10. Verified in the S3 console that the state file was available at ec2-httpd/terraform.tfstate.

Code — Initial backend resource configuration

This configuration is used separately to create the S3 bucket and DynamoDB table. If you already created both resources, you do not need to apply it again.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.70.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "terraform_state" {
  bucket = "YOUR-GLOBALLY-UNIQUE-BUCKET-NAME"

  tags = {
    Name = "Terraform-State-Bucket"
  }
}

resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
```

```
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state_encryption" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "state_public_access" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_dynamodb_table" "terraform_lock" {
  name         = "terraform-state-lock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
```

```
  }

  tags = {
    Name = "Terraform-State-Lock"
  }
}

output "state_bucket_name" {
  value = aws_s3_bucket.terraform_state.bucket
}

output "lock_table_name" {
  value = aws_dynamodb_table.terraform_lock.name
}
```

Replace YOUR-GLOBALLY-UNIQUE-BUCKET-NAME with your actual bucket name before applying this initial configuration.

Code — S3 backend in the existing Question 3 main.tf

Add the following backend "s3" block inside the existing terraform block. Keep the provider, security group, EC2 instance and outputs from Question 3 unchanged.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.70.0"
    }
  }

  backend "s3" {
    bucket         = "YOUR-ACTUAL-S3-BUCKET-NAME"
```

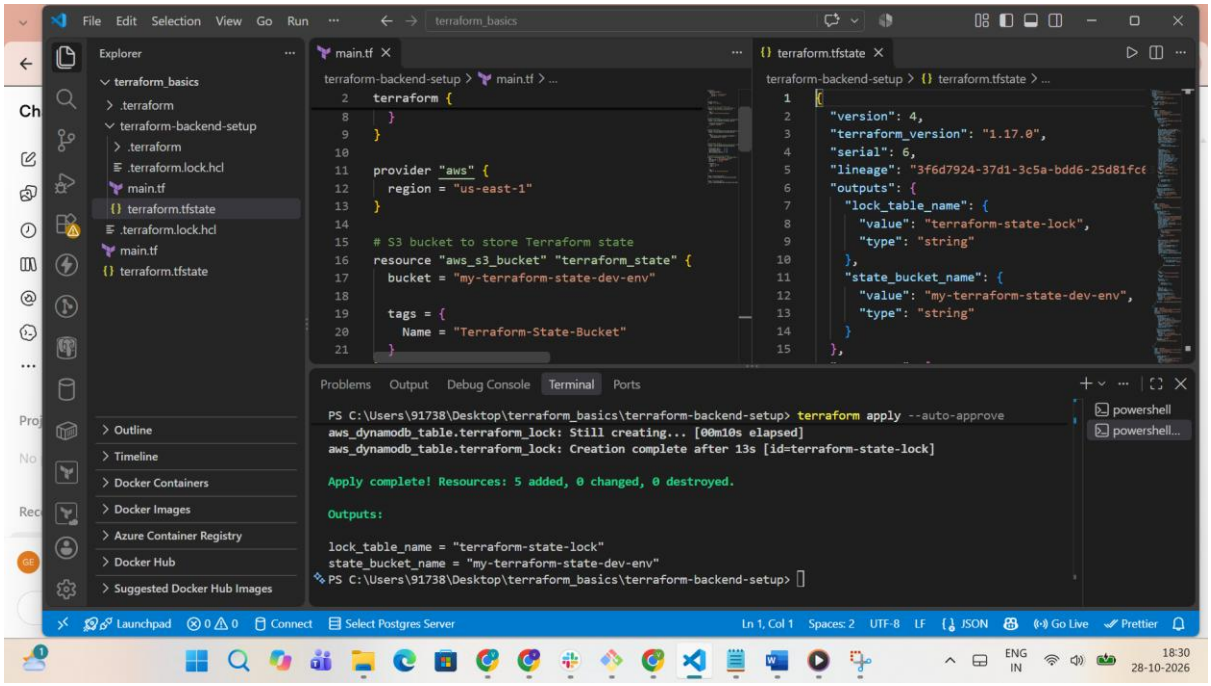
```
  key          = "ec2-httpd/terraform.tfstate"
  region       = "us-east-1"
 }
}
```

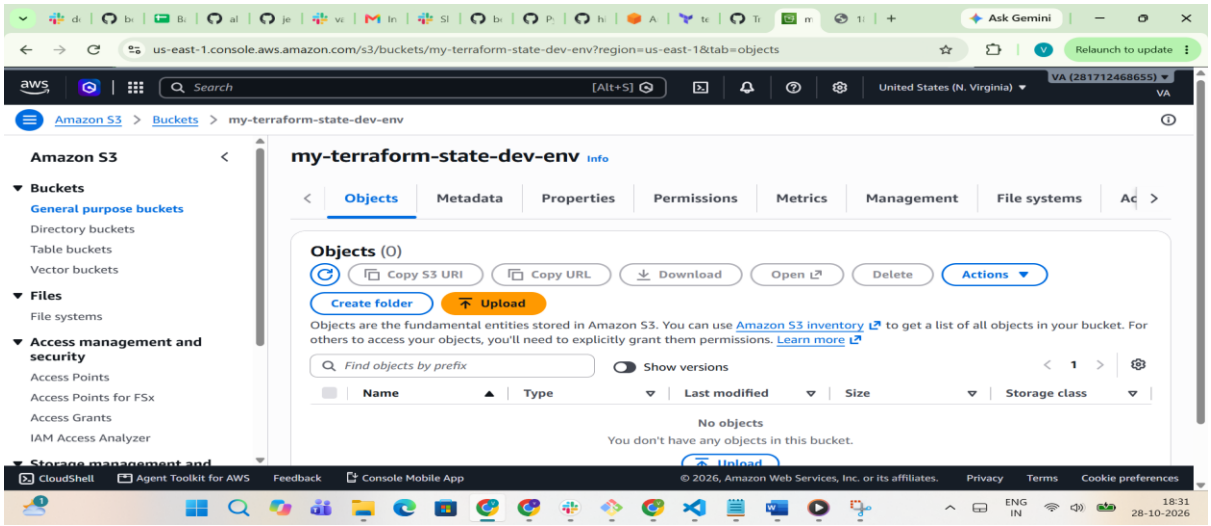
Commands executed

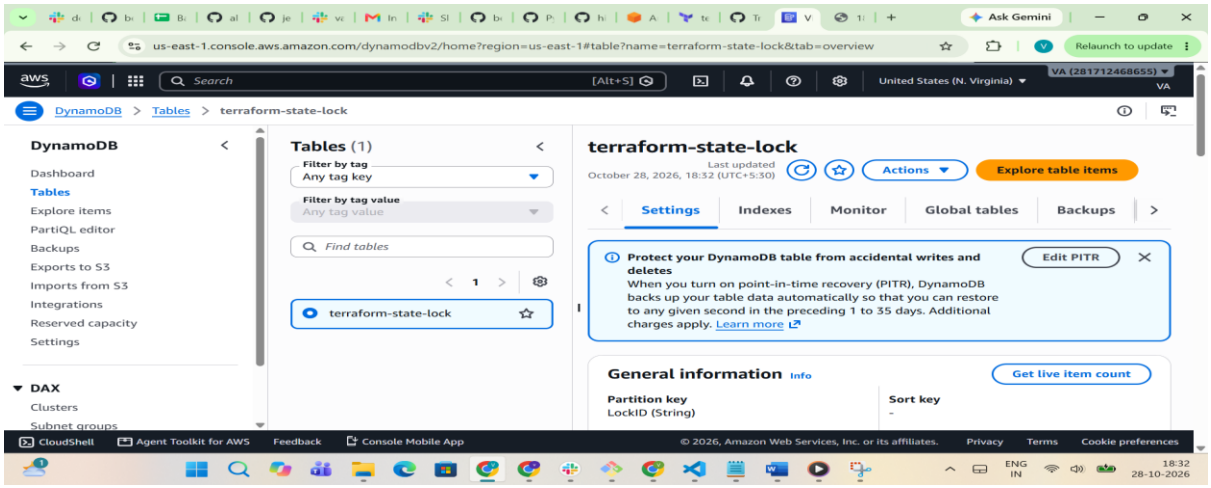
terraform init -migrate-state

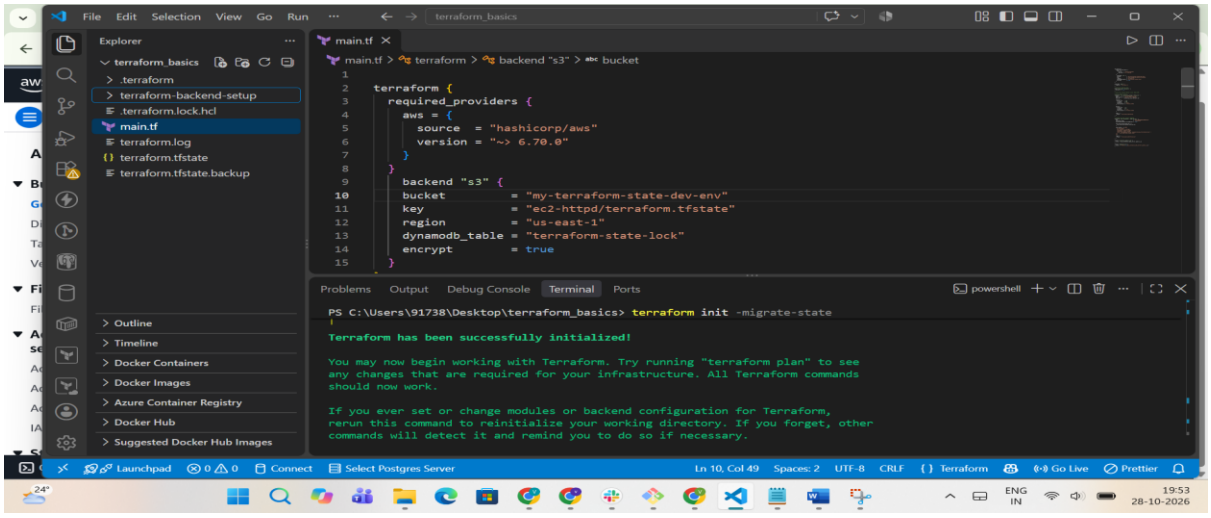
terraform apply

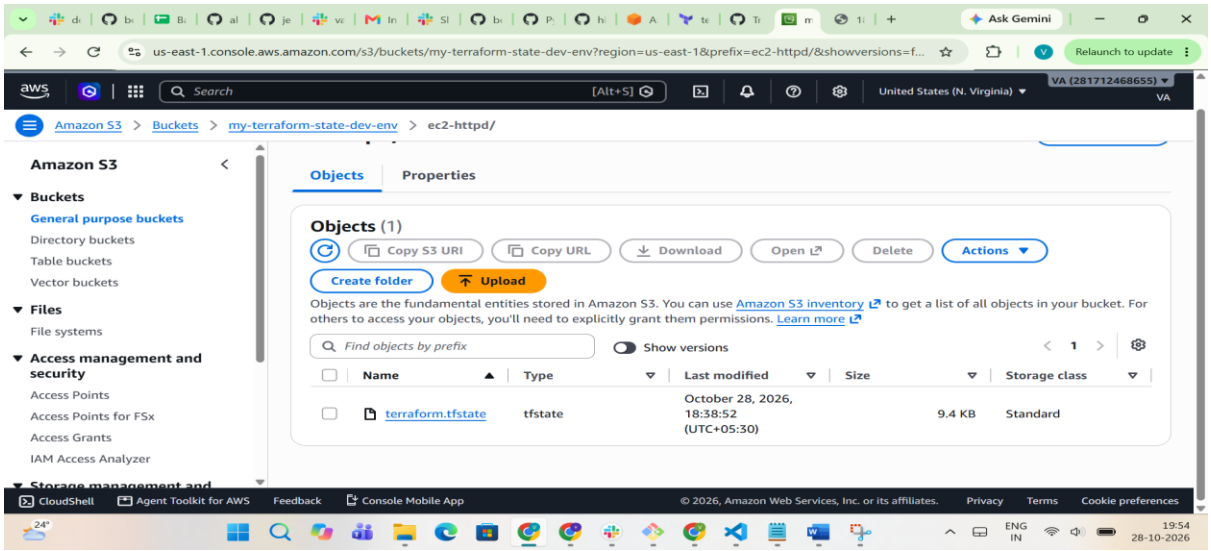
Result: The existing EC2 instance continued to be managed by Terraform, and its state file was migrated to the S3 backend.











5). Set up **DynamoDB locking** for task 3.

Implementation steps:

1. Opened the AWS Management Console and navigated to DynamoDB.

2. Created a DynamoDB table named terraform-state-lock in the us-east-1 region.
3. Configured LockID as the partition key with the data type String.
4. Selected the PAY_PER_REQUEST billing mode.
5. Opened the existing terraform_basics\main.tf file.
6. Added the dynamodb_table setting to the existing S3 backend configuration.
7. Confirmed that the S3 bucket, DynamoDB table and Terraform configuration used the same AWS region.
8. Executed terraform init to initialize the backend configuration.
9. Executed terraform validate and terraform plan to verify the configuration.
10. Tested state locking by starting a Terraform operation in one PowerShell terminal and attempting another Terraform operation in a second terminal.
11. Verified that the second operation could not acquire the state lock while the first operation held it.
12. Cancelled the first operation and verified that Terraform released the lock.
13. Checked the DynamoDB table to observe the temporary lock record during an active operation, when visible.

Code — DynamoDB locking configuration

The locking configuration is included in the same S3 backend block in the existing main.tf file.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.70.0"
    }
  }

  backend "s3" {
    bucket         = "YOUR-ACTUAL-S3-BUCKET-NAME"
    key            = "ec2-httpd/terraform.tfstate"
```

```
  region         = "us-east-1"
  dynamodb_table = "terraform-state-lock"
  encrypt        = true
 }
}
```

